2020 5th International Conference on Advanced
Robotics and Mechatronics (ICARM)

Development of Tracking and Control System Based on Computer Vision for RoboMaster Competition Robot

Xinyang Tang, Chuntao Leng, Yiheng Guan, Li Hao, Shukun Wu

***Abstract*—In this paper, we present a tracking and control system used to auto-aiming and shooting some given targets. By combining the traditional computer vision method and the neutral network method, our detection for some specific objects achieves real-time (100fps+) and it is robust enough to work on most normal situations. To achieve better performance in shooting moving targets, we fuse the IMU data to do the motion prediction. This makes the system outperforms the human players in hit rate. This system is applied for the RoboMaster competition as an aim-support system and shows good results.**

I. Introduction

As the improvement of the robot technic, the computer vision control system plays a more and more important role. The computer vision can play a different role in the control system, including SLAM, tracking system and so on.

RoboMaster competition is a National University Robot Competition in China. More than 200 teams participated in the competition, and many foreign teams also participated. The participations need to develop their own robots from mechanical structure to control system. To win this competition, players needs to shoot the specific part of the enemy with rubber bullets. Computer vision plays a very important place in this competition. So how to develop an efficient visual tracking system has become an important part of this competition.

BU Lingzheng et al. [1] introduce digital classification in RoboMaster2018. This is a part of the competition. Other researches about object detection like Moving Object Detection[2], they mainly base on a static background, which can only detect moving object while camera itself is not moving. However, in RoboMaster competition, the camera always keeps moving. Object detection base on neutral network like SSD[3], YOLO[4,5] do preform pretty will. However, to achieve better controlling performance, the inference time do matter a lot, these methods do not that fast, (<30fps on NVIDIA TX2). Traditional methods based on image pyramid and sliding window use HOG[6] or SIFT[7], they need a template first.

This paper takes RoboMaster competition as the research background and intends to develop a real-time and robust computer vision control system. We achieve low latency (>100fps). And to achieve better controlling performance, we also use IMU data and do motion prediction, which will improve the hit rate.

*Research supported by National Natural Science Foundation of China (Grant No. 51605283).

Xinyang Tang is with the School of Electronic Information and Electrical Engineering, Shanghai Jiao Tong University, Shanghai, China (e-mail: lloi_yang@sjtu.edu.cn).

Chuntao Leng (corresponding author) is with Student Innovation Center, Shanghai Jiao Tong University, Shanghai, China (phone: 0086-21-54743188; e-mail: ctleng@sjtu.edu.cn).

Yiheng Guan is with the School of Mechanical Engineering, Shanghai Jiao Tong University, Shanghai, China (e-mail: 285846897@qq.com).

Li Hao is with Student Innovation Center, Shanghai Jiao Tong University, Shanghai, China (e-mail: haoli_alicia@sjtu.edu.cn).

Shukun Wu is with Student Innovation Center, Shanghai Jiao Tong University, Shanghai, China (e-mail: wsk798@sjtu.edu.cn).

II. Scene and Challenges

A. RoboMaster Competition Area

The competition area of RoboMaster2019 is a little dim and has many lights for decoration, which makes the detection difficult. Also, the competition area is large, so that the target in camera will be small most of the time, in figure 1.

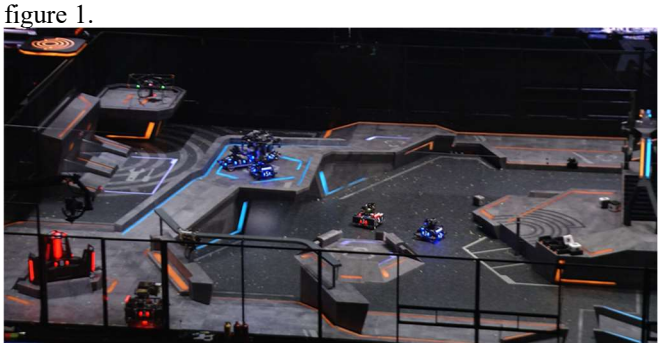
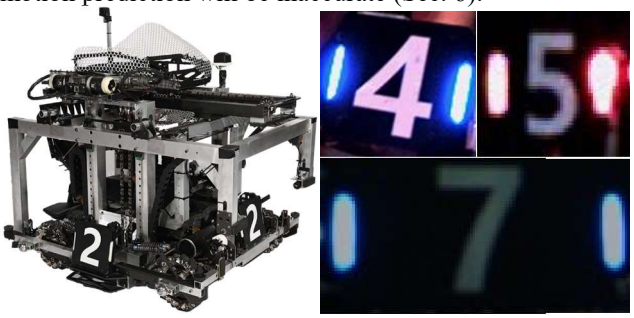


Fig. 1: The competition area. *The decoration lights in the competition area could be misleading. The area is large, about a basketball court.*

B. The Object Detection Requirement in RoboMaster

Our detection target is an armor module on the enemy's robots. There are two different kinds of armor module, the big armor (230mm*127mm) and the small armor (135mm*125mm), in figure 2. Both of them are stuck a digital sticker (from 1 to 8, without 6), and have two long lights which bright red or blue on the edge of the armor module. When we are the red team, the lights on our robots will be red, and the lights on opponent's robots will be blue. The digital sticker on armor module is used to distinguish between different robots. Different robots have different digital sticker on armor module.

Also, the detection needs to be real-time enough, otherwise if the detection has a high latency (>30ms), the motion prediction will be inaccurate (Sec. 6).



(1) (2)

Fig 2: Different kinds of armor modules. *(1) The No.2 engineer robot and its armor modules. (2) The blue No.4 armor module on the top left. The*

978-1-7281-6479-3/20/$31.00 ©2020 IEEE

Authorized licensed use limited to: Purdue University. Downloaded on October 25,2026 at 03:38:59 UTC from IEEE Xplore. Restrictions apply.

red No.5 armor module on the top right. The blue No.7 armor module on the bottom.

C. The Necessity of Motion Prediction

Because the component's robot is usual not static but moving, and the bullet need time to fly, just pointing where target is now is not enough, in figure 3.

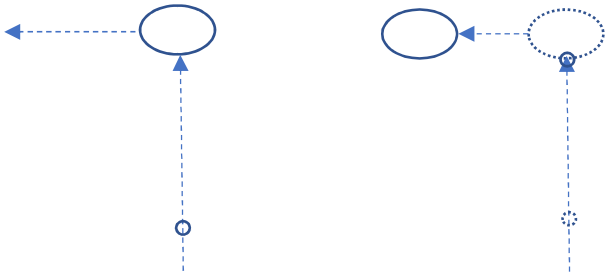


Fig. 3: Shooting without motion prediction. *In the left figure, we shoot a bullet. But in the right figure, when the bullet reaches where the target used to be, the target is no longer there.*

D. Challenges Summary

(1) Real-time detection to achieve a high-performance control and motion prediction.

(2) The targets are small most of the time, so we need to detect those small targets.

(3) As less as background detection as possible. Because of the decoration lights, background detection is more likely to appear. However, the background detection may make the players confused.

(4) Motion prediction to increase the hit rate.

III. System Development

A. Control System Framework

The robot consists of a chassis with armor modules and a gimbal with shooting device and camera device. The compute device is hidden to avoid being hit, in figure 4.

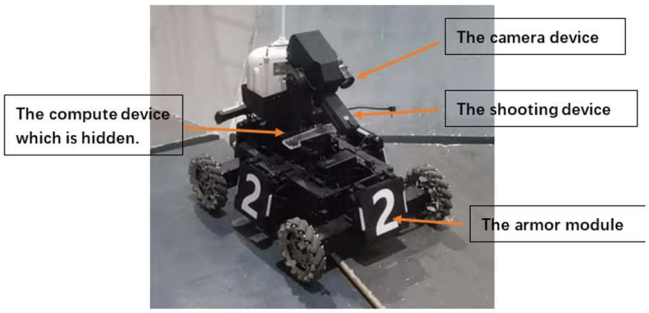


Fig. 4: The picture of our robot.

The control framework consists of two main part. The host, a mini compute device, detects the targets and does motion prediction. Then it sends the control information to the slave. The slave, a MCU, receives the control information and controls the motor to move, in figure 5.

This paper introduces the host part.

B. The Brief Flow Path of Our System

To achieve the requirements of real-time (100fps+) and robustness, and considering that the objects we need to detect have an obvious feature, the two lights, we decide to use the contours-based method with OpenCV to find the candidates, and use Convolutional Neural Network (CNN) to screen the candidates, finding the real targets, and getting the number on the armor at the same time.

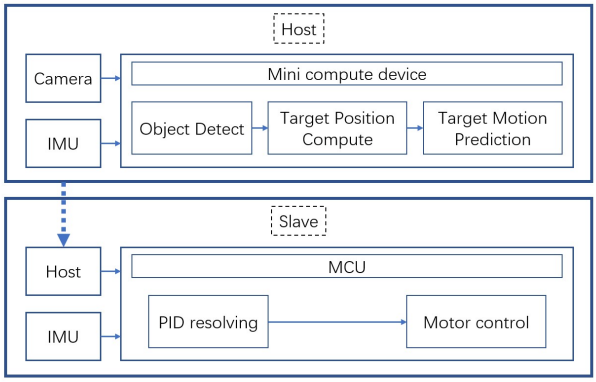


Fig. 5: The framework of our auto-system.

The flow path is shown blow, in figure 6:

(1) First, do the preprocess on the image. We have a priori knowledge that each target has two lights, and light is much brighter than the background, so we extract the blue channel or red channel of the image according to the opponent's color. Then binarize this channel. In order to decrease the influence of the noise, then we erode and dilate the binary image.

(2) Then find all the contours on the binary image, and filter them base on the geometric features of the light. The contours must be a long and thin rectangle or oval.

(3) After find all possible lights, we match different pair of lights also according to the geometric features of the armor, like the distance between two lights, the angle of two lights, length-width ratio, and so on. So far, we get some candidates, and many other teams who participate RoboMaster choose one candidate as the target and send control information to the micro control unit (MCU).

(4) However, the candidates many contain miss detection. We use the CNN classifier (Sec. 4) to check the candidates and identify the digital sticker on the armor. Then choose one as the target base on the priority.

(5) After find the target, we calculate the relative spherical coordinate of the target. With the relative spherical coordinate, we can control the movement of the muzzle. However, the rubber bullets need time to reach the armor module, so we do the motion prediction of the target. Calculate the absolute spherical coordinate of the target base on the IMU data (Sec. 5). Then calculate the velocity of the target base on the difference of the target coordinate. Finally, predict the position (Sec. 6) of the target and send control information.

(6) Sometimes, the opponent's robot will move in a different way, which the chassis will rotate continuously to avoid bullets. Such motion is different, so that we design a specific model (Sec. 6) to deal with this situation.

Because the first 3 procedures of the whole flow path are some common procedures, the following paper will start from the fourth procedure, the classifier.

978-1-7281-6479-3/20/$31.00 ©2020 IEEE

Authorized licensed use limited to: Purdue University. Downloaded on October 25,2026 at 03:38:59 UTC from IEEE Xplore. Restrictions apply.

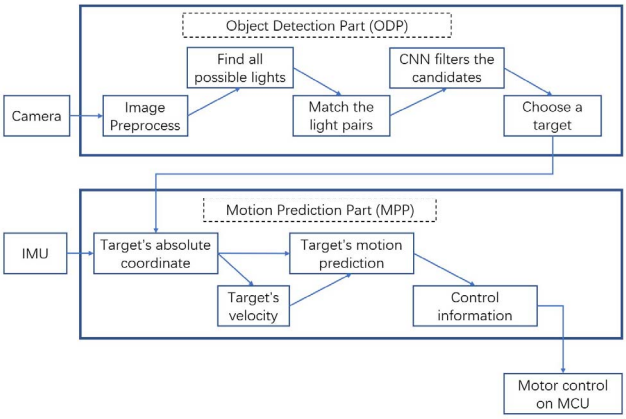


Fig. 6: The flow path of our system. *The system can be split into two main parts, the object detection part (ODP) and the motion prediction part (MPP). ODP provides the targets' informations to MPP. MPP calculate the future position of each targets, and then send a control information to the MCU.*

IV. The Choice and the Designing of the Classifier

A. The Common Classifier

SVM [8] is a very classical classifier. But if want to use it for image classification, we need to use man-made features to extract the feature vector of the image firstly, which makes the precision highly depends on the features' quality, and is less robust.

However, CNN based classifier is trained end-to-end, which does not need man-made features, and performs much better than SVM. LeNet[9] is also a classical image classifier, it was designed for low resolution handwriting digits, which is accurate and fast. Other CNN classifiers like Resnet or GoogLeNet are designed for large resolution images, and have a high inference latency, so we will not choose them.

B. Our Classifier

To be real-time, the classifier must be fast; to be robust, the classifier must be accurate. Fortunately, LeNet matches our needs very well, Both for low resolution digit image recognition.

The classifier has 15 classes, 1 background + 2 color * 7 number. The input images' resolution is 48*36.

When the resolution of input image is 48*36, the inference time is 0.6ms on average, which can be real-time.

C. Training of the Classifier

Because the main purpose of this classifier is to reject mis-detect, we over-sample the negative samples. To extend the sample size, we use the ImageNet dataset, train those images as negative samples.

To reduce the time spent on labeling the images, we try a new method.

(1) Label a few numbers of images, about 1000.
(2) Train the model
(3) Apply the model on more images, and select the wrong classification manually.
(4) If the precision reaches the requirement, end the procedure.
(5) Else, add the wrong classifying image to the dataset and go to (2).

We train the classification models with about 20000 images, and validate them with other 1000 images. Then calculate the precisions. The images include different orientation, lighting and size. The result shows that LeNet outperforms the SVM. We decide to use LeNet finally.

TABLE I. The Classification Presicion of Defferent Mdels

Number	Classifier Models	
	LeNet	***SVM***
1	99.2%	92.7%
2	98.6%	93.5%
3	99.6%	95.2%
4	99.3%	94.9%
5	99.0%	95.9%
7	98.2%	92.1%
8	99.1%	96.0%
Latency	0.6ms	<0.1ms

V. Absolute Spherical Coordinates of the Target

A. Relative Spherical Coordinates

We use the theory of pinhole imaging to solve the relative spherical coordinates, in figure 7.

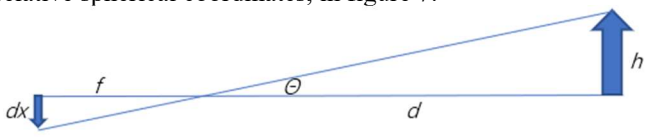


Fig. 7: Pinhole imaging. θ *is the angle in spherical coordinates.* ***f*** *is the focus of the camera.* ***dx*** *is the distance of the center of the target on the image to the center of the image or the height of the target on the image.* ***d*** *is the distance of the target.* ***h*** *is the height of the target.*

As is shown in the figure, we can know that

$$\theta = \arctan\left(\frac{dx}{f}\right) \tag{1}$$

The focus can be calibrated in advance. So, the angle of spherical coordinates is not a matter.

To calculate the distance of the target, we use the theory of pinhole imaging as well. Because the armor module has a constant height, and always perpendicular to the ground, the distance can be calculated easily.

$$d = \frac{f * h}{dx} \tag{2}$$

In (2), the numerator $f * h$ is a constant value which can be calibrated in advance. So, the distance can be calculated.

B. IMU Data and Absolute Spherical Coordinates

To calculate the absolute spherical coordinates, we need to know the attitude angle of the camera, which can be measured from the IMU. However, the data frame rate from camera is about 100Hz, and the data frame rate from IMU is about 400Hz. They do not share a same time stamp, which will bring some error to the system, in figure 8.

The VINS [10] told about the fusion of camera and IMU, its purpose is stronger and is uses a relatively complex method to analyses.

Because our purpose of using IMU is pure and simple, to simplify the procedure, we use an easy method, which can still meet our requirements.

Authorized licensed use limited to: Purdue University. Downloaded on October 25,2026 at 03:38:59 UTC from IEEE Xplore. Restrictions apply.

There is a time difference between two data which should be a constant value. To calibrate the time difference, we use a simple way.

(1) Put an armor module which is the target on the ground, keep it motionless.

(2) Pre-set a man-made time difference.

(3) Use a camera to detect this target, shaking the camera in the same time and printing the absolute angle calculate by the program.

(4) If the error of the absolute angle is small enough, end this procedure.

(5) Else, change the time difference and go to (3).

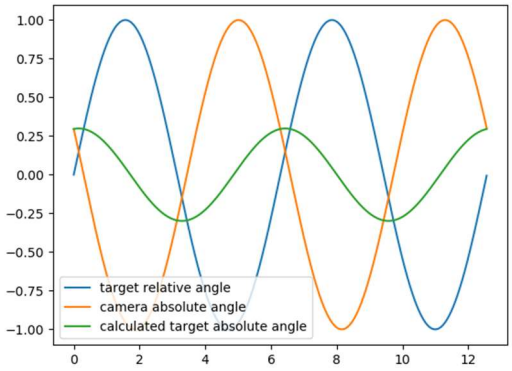


Fig. 8: Error due to the time stamp. *In this figure, there are two kinds of data, camera attitude angle from IMU (the orange line) and relative angle from program (the blue line). The misalignment of time stamp will bring error to the absolute angle (the green line).*

VI. Motion Prediction

A. Motion Prediction in Normal Situation

To do motion prediction, the most important thing is to calculate the velocity of the target. To simplify the calculation, we suppose that target is moving with a constant velocity.

Let x_i be the angle sequence of targets, t_i be the time stamp respectively. v_i be the velocity of target, px_i be the predicted angle, T be the predicted time length, which may be 100ms to 500ms.

$$v_i = \frac{x_i - x_{i-1}}{t_i - t_{i-1}} \tag{3}$$

$$px_i = x_i + v_i * T \tag{4}$$

However, there some factor will influence the precision of motion prediction.

(1) there is a noise e_i on the angle of targets. Suppose that $e_i \sim N(0,1)$

$$x_i' = x_i + e_i \tag{5}$$

$$v_i' = \frac{x_i' - x_{i-1}'}{t_i - t_{i-1}} = v_i + {e^{(1)}}_i \qquad e_i^{(1)} \sim N\left(0, \frac{2}{t_i - t_{i-1}}\right) \tag{6}$$

$$px_i' = x_i' + {v'}_i * T = x_i + v_i * T + e_i^{(2)} \qquad e_i^{(2)} \sim N\left(0,1 + \frac{2T}{t_i - t_{i-1}}\right) \tag{7}$$

In (7), we know that $\frac{T}{t_i - t_{i-1}} > 10$ usually, so the error of px_i' can be large. To reduce the error of px_i', we use the Kalman filter.

(2) Because we suppose that the target is moving with a constant velocity, when they do not move this way, there will also be an error.

$$px_i^{real} = x_i^{real} + v_i^{real} * T + \frac{1}{2} a_i^{real} * T^2 \tag{8}$$

$$px_i^{real} - px_i = \left(x_i^{real} - x_i\right) + \left(v_i^{real} - v_i\right) * T + \frac{1}{2} a_i^{real} * T^2 \tag{9}$$

Suppose that our measurement is accurate.

$$x_i^{real} = x_i \tag{10}$$

$$v_i^{real} = v_i \tag{11}$$

$$px_i^{real} - px_i = \frac{1}{2} a_i^{real} * T^2 \tag{12}$$

During the competition, $a_{i-max}^{real} \approx 8\text{m/s}$, $T = \frac{d}{v_{bullet}} = \frac{4m}{25m/s} = 0.16s$, most of the time. So that $px_i^{real} - px_i \approx 0.1024\text{m} = 10.24\text{cm}$. The width of big armor module is 23.0cm, which is smaller than 10.24cm*2. This means this error can be ignored, because is will not influence the hit rate so much.

B. Motion Prediction in Special Situation

There is a special situation, in figure 9, that the opponent's robot will rotate its chassis continuously, in order to avoid being hit. The angular velocity of the rotation can be very high (540°/s). The valid hit and detect angle range is about 90°, so the time from one armor appear to disappear is only about 167ms. If the predict time T > 167ms, we will never hit this armor.

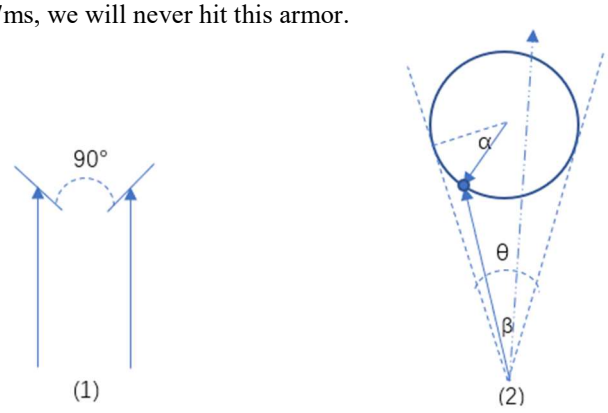


Fig. 9: Motion Prediction in Special Situation. *(1) illustrate the valid hit and detect angle. The program is hard to detect the target armor module and bullet is hard to hit the armor as well out of this angle range. (2) This is a diagram of this special situation. At the bottom of this figure is our camera, and the dotted arrow is the central axis of out camera. The big circle on the upper side of this figure is the path of one of the armor modules on the opponent's robot and the small dot on the circle is the place of the armor module.* α *is the angle the armor turns, and* β *is relative angle measured by the vision program.*

Let δ be the angle difference between the central axis and the rotation center, r be the radius of rotation, l be the distance between the rotation center and our camera. According to The Law of Sines

Authorized licensed use limited to: Purdue University. Downloaded on October 25,2026 at 03:38:59 UTC from IEEE Xplore. Restrictions apply.

$$\frac{l-r}{sin\left(arccos\left(\frac{r}{l}\right)-\alpha\right)}=\frac{\mathrm{r}}{\sin(\beta-\delta)} \tag{13}$$

Simplify the equation above, we get $\alpha \propto \beta$. Suppose that opponent's robot only rotates without translation and rotates with a constant angular velocity. We get $\beta \propto \alpha \propto t$.

Let β_i be the relative angle sequence of one valid angle range, t_i be the time stamp respectively.

$$t = \hat{k}\beta + \hat{b} \tag{14}$$

$$\hat{k} = \frac{\sum \beta t - \frac{1}{N}\sum \beta \sum t}{\sum \beta^2 - \frac{1}{N}(\sum \beta)^2} \tag{15}$$

$$\hat{b} = \bar{t} - \hat{h}\bar{x} \tag{16}$$

From the equation above we can calculate $t^{(i)}_{\beta^{(i)}=0}$, which is the time stamp when the armor module is in front of our camera. When the next armor module turns to the front of our camera, we can also calculate $t^{(i+1)}_{\beta^{(i+1)}=0}$ in the same way, and the armor turn 90°, so the angular velocity can be calculated as well.

$$\omega = \frac{90^\circ}{t^{(i+1)}_{\beta^{(i+1)}=0} - t^{(i)}_{\beta^{(i)}=0}} \tag{17}$$

When we know the ω, we can predict when next armor will be in front of our camera, and shoot in advance.

$$t'^{(i+2)}_{\beta^{(i+2)}=0} = t^{(i+1)}_{\beta^{(i+1)}=0} + \frac{90^\circ}{\omega} \tag{18}$$

C. *The Application of Motion Prediction*

After motion prediction procedure, we know where the target will be in the future. Then the host will send this future coordinate to the slave MCU, and the MCU will control the shooting device points this future coordinate. If we fire this time, the bullet will hit the target.

VII. Experiments and Results

To test our auto-system, we give the robot 200 rubber bullet, and 2 minutes to shoot, in figure 10. We test the hit rate of different moving pattern, and compared with the manual control robot.



Fig. 10: Experimental scene. *On the top left of this figure, there is a guard robot, and on the bottom right of the figure, there two robots shooting the guard robot.*

TABLE II. Hit Rate of 200 Bullets

Way of hit	Target and its movement		
	Guard	Normal situation	Special situation
Our auto-system	91%	35.5%	85.5%
Manual	60%	19.5%	48%

Guard is a special robot which can only move on fixed orbit automatically, so its movement is almost with a constant velocity which highly match our prediction model. It reaches the highest hit rate no doubting. The normal situation means the robot keep moving randomly. This situation does not match the two predict models above, so it reaches the lowest hit rate. The special situation means the robot keep rotating its chassis without translation. This situation matches the second model pretty well, so it also reaches a high hit rate. Both of these situations, our auto-system outperforms the manual control.

In the table above, we can conclude that the most important condition which influence the hit rate is the moving pattern of targets. Because our motion prediction has some assumptions, the performance will be the best when the assumptions match the realities.

To easily check the detection procedures and find the fault quickly, we visualize the main procedures, in figure 11.

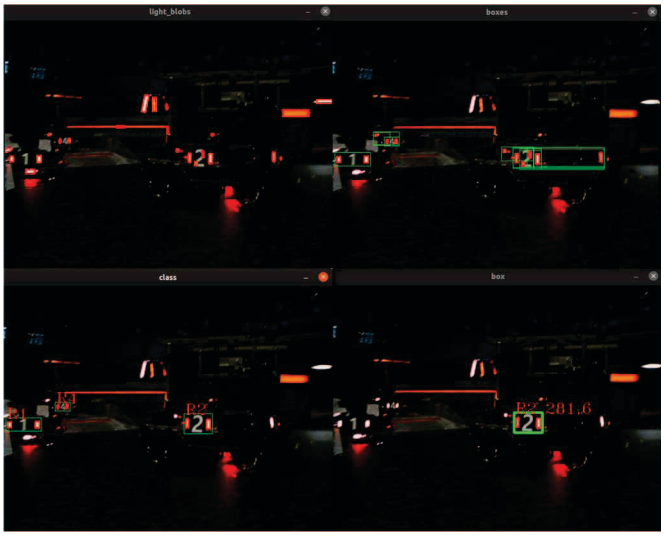


Fig. 11: Detection procedure. *From top left to the bottom right, they are lights detection, light pairs matching, CNN filters and target choosing by priority, respectively.*

VIII. Conclusions

With the computer vision and motion prediction method, our auto-system can detect and track the specific targets and shoot them with rubber bullet.

The key points to our system are real-time (due to the edge-based way of detection), less background detection (due to the classifier), easily dealing with the small targets (due to edge base way of detection) and better hit rate (due to motion prediction).

We applied this control system to RoboMaster competition. Nearly 200 teams from all over the world participated in the competition, and our Jiaolong robot team won the second place in RoboMaster 2019.

978-1-7281-6479-3/20/$31.00 ©2020 IEEE

Authorized licensed use limited to: Purdue University. Downloaded on October 25,2026 at 03:38:59 UTC from IEEE Xplore. Restrictions apply.

The future work will be trying to further improve the hit rate, including the detection when light is special and the convergence time of prediction.

ACKNOWLEDGMENT

This work was funded by National Natural Science Foundation of China (Grant No. 51605283).

REFERENCES

[1] L. Bu, et al. "Multi-source digit recognition algorithm based on improved convolutional neural network," *Journal of Computer Applications*, vol. 38, no. 12, pp. 3403-3408, May 2018.

[2] A. Sumaya, E. S. Khaled, and E. Shireen. "Moving object detection in spatial domain using background removal techniques - state-of-art," *Recent Patents on Computer Science*, vol. 1, no. 1, pp. 32-54, 2010.

[3] W. Liu, et al. "SSD: single shot multibox detector," *European conference on computer vision*, 2016, pp. 21-37.

[4] J. Redmon, et al. "You only look once: unified, real-time object detection," *The IEEE Conference on Computer Vision and Pattern Recognition,* Nov. 2015, pp. 779-788.

[5] J. Redmon, and A. Farhadi. "YOLO9000: better, faster, stronger," *The IEEE Conference on Computer Vision and Pattern Recognition*. 2017, pp. 6517-6525.

[6] N. Dalal, and B. Triggs. "Histograms of oriented gradients for human detection," *IEEE Computer Society Conference on Computer Vision & Pattern Recognition*, vol. 56, no. 3, pp. 364-371, Aug. 2005.

[7] D. G. Lowe, "Distinctive image features from scale-invariant keypoints," *International Journal of Computer Vision*, vol. 20, no. 2, pp. 91-110, 2004.

[8] Z. Q. Zeng, et al. "Fast training support vector machines using parallel sequential minimal optimization", *2008 IEEE 3rd international conference on intelligent system and knowledge engineering*, 2008, vol. 1, pp. 997-1001.

[9] Y. Lecun, et al. "Gradient-based learning applied to document recognition," *Proceedings of the IEEE*, vol. 86, no. 11, pp. 2278-2324, 1998.

[10] T. Qin, P. Li, and S. Shen. "VINS-Mono: a robust and versatile monocular visual-inertial state estimator," *IEEE Transactions on Robotics*, vol. 34, no. 4, pp. 1004-1020, 2017.

Authorized licensed use limited to: Purdue University. Downloaded on October 25,2026 at 03:38:59 UTC from IEEE Xplore. Restrictions apply.